

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: ПОИСК ПОДСТРОКИ В СТРОКЕ

Студент гр. 4345

Пикалев Н.Г.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Изучить и реализовать алгоритм Кнута-Морриса-Пратта для поиска подстроки в строке.

Задание.

1. Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 2500$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T .

Вход:

- Первая строка – P
- Вторая строка – T

Выход:

Индексы начал вхождений P в T , разделённые запятой; если P не входит в T , то вывести -1.

2. Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$).

Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

- Первая строка – A
- Вторая строка – B

Выход:

Если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Выполнение работы.

При наивном решении задачи поиска подстроки в строке естественным является алгоритм с асимптотикой $O(N * M)$, где N — длина текста, m — длина шаблона. Такая сложность возникает из-за того, что для каждой позиции текста производится посимвольное сравнение шаблона целиком. При этом предыдущие результаты сравнений не учитываются, что приводит к избыточным вычислениям.

Оптимизировать наивный метод поиска подстроки можно с помощью алгоритма Кнута-Морриса-Пратта. Основой алгоритма является префикс-функция. Для каждой позиции i в строке-шаблоне префикс-функция $\pi[i]$ определяет длину самого длинного собственного префикса подстроки $P[0...i]$, который одновременно является ее суффиксом. Префикс-функция позволяет избежать избыточных сравнений и возвратов к уже пройденным символам основной строки.

Для выполнения первой части работы был создан файл `kmp1.py`. Функция `kmp_search` принимает на вход строку-шаблон P и исходную строку T . В процессе работы функции происходит вычисление массива префикс-функции π , который позволяет определить, на какую позицию должен отойти указатель в шаблоне при несовпадении символов. В основном цикле поиска переменная i отвечает за текущую позицию в исходной строке T , j — за позицию в шаблоне P . При совпадении символов значения j увеличивается на 1, при несовпадении происходит откат по префикс-функции. Если длина совпавшей части j достигает длины шаблона, это означает, что найдено вхождение, и его начальный индекс $(i - \text{len}(P) + 1)$ сохраняется. В конце работы программа выводит все найденные индексы через запятую, а если вхождений не найдено — выводит -1.

Для выполнения второй части работы был создан файл `kmp2.py`. Была написана функция `kmp_search`, которая проверяет, является ли строка A циклическим сдвигом строки B . В начале функция проверяет равенство длин

строк; если они разной длины — возвращает -1. Если обе строки пустые — возвращает 0. Далее строится префикс-функция для строки В (при этом в коде используется длина А, которая равна длине В, поэтому массив p_i имеет размер $\text{len}(A)$). После построения префикс-функции выполняется поиск строки А внутри удвоенной строки (В + В). Чтобы не создавать новую строку в памяти для длинных входных данных, используется циклический доступ к символам А через оператор $i \% \text{len}(A)$. Если в процессе поиска счётчик совпавших символов j достигает длины $\text{len}(A)$, это означает, что строка А найдена как подстрока в В + В, а следовательно, А является циклическим сдвигом В. Функция вычисляет индекс начала сдвига ($i - \text{len}(A) + 1$) и возвращает его. Если сдвиг не найден, возвращается -1. В основной части программы происходит ввод строк А и В, вызов функции и вывод результата.

Затраты алгоритма по времени: $O(N * M)$. N – длина основной строки, M – длина подстроки.

Затраты алгоритма по памяти: $O(N * M)$ из-за хранения строк и массива префикс-функции.

Исходный код программ см. в приложении А.

Результаты тестирований программ см. в таблице 1 и 2.

Таблица 1 – Тестирование программы kmp1.py

Ввод	Результат	Комментарий
ab abab	0,2	Пример из условия первой задачи
defabc abcdef	-1	Пример из условия второй задачи
abcdefsdx def	-1	Если первая строка длиннее второй
ww dfewwwsddwwwsda	3,4,5,10,11	Несколько найденных подстрок

Таблица 2 – Тестирование программы kmp2.py

Ввод	Результат	Комментарий
------	-----------	-------------

ab abab	-1	Пример из условия первой задачи
defabc abcdef	3	Пример из условия второй задачи
qwerty ytrewq	-1	Пример без циклического сдвига
qwerty rtyqwe	3	Пример с циклическим сдвигом

Выводы.

В ходе выполнения данной лабораторной работы был изучен и реализован алгоритм Кнута-Морриса-Пратта для эффективного поиска подстроки в строке.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: kmp1.py

```
def kmp_search(P, T):
    lenP = len(P)
    Pi = [0] * lenP
    j = 0
    print(f"\nЗначения префикс-функции для {P}\n")

    for i in range(1, lenP):
        while j > 0 and P[i] != P[j]:
            j = Pi[j - 1]

        if P[i] == P[j]:
            j += 1

        Pi[i] = j

    lenT = len(T)
    result = []
    j = 0

    for i in range(lenT):
        print(f"Символы: t[{i}] = {T[i]} и p[{j}] = {P[j]}")

        while j > 0 and T[i] != P[j]:
            print(f"Символы не совпали, идем назад по префикс-
функции, новый j: {Pi[j - 1]}")
            j = Pi[j-1]

        if T[i] == P[j]:
            print(f"Символы совпали: {T[i]} и {P[j]}")
            j += 1
            print(f"Идем вперед, новый j: {j}")

        else:
            print(f"Символы не совпали: {T[i]} и {P[j]}")

        if j == lenP:
            pos = i - lenP + 1
            print(f"Найдена полная подстрока {P} на индексе
{pos}")

            result.append(str(pos))
            j = Pi[j - 1]
            print(f"Идем назад по префикс-функции, новый j:
{j}\n")

    return result

P = input("Первая строка: ")
T = input("Вторая строка: ")

result = kmp_search(P, T)
```



```

if result:
    print(f"Результат: {' '.join(map(str, result))}")

else:
    print(f"Результата нет: {-1}")

```

Название файла: kmp2.py

```

def kmp_search(A, B):
    lenA = len(A)

    if lenA != len(B):
        return -1

    if lenA == 0:
        return 0

    pi = [0] * lenA
    j = 0
    print(f"\nПостроение префикс-функции для {A}\n")

    for i in range(1, lenA):
        print(f"Шаг i={i}: A[{i}]='{A[i]}' , A[{j}]='{A[j]}'")
        while j > 0 and B[i] != B[j]:
            j = pi[j - 1]
            print(f"Символы не совпали, идем назад по префикс-
функции, новый j={j}")

        if B[i] == B[j]:
            j += 1
            print(f"Символы совпали, идем вперед, новый j={j}")

        pi[i] = j
        print(f"pi[{i}]={j}\n")

    j = 0

    for i in range(2 * lenA - 1):
        while j > 0 and A[i % lenA] != B[j]:
            j = pi[j - 1]
            print(f"Идем назад по префикс-функции, новый j={j}")

        if A[i % lenA] != B[j]:
            print(f"A[{i % lenA}]='{A[i % lenA]}' и
B[{j}]='{B[j]}' не совпали")

        if A[i % lenA] == B[j]:
            print(f"A[{i % lenA}]='{A[i % lenA]}' и
B[{j}]='{B[j]}' совпали")
            j += 1
            print(f"Идем вперед по префикс-функции, новый
j={j}\n")

        if j == lenA:
            return i - lenA + 1

```

```
        return -1

A = input("Первая строка: ")
B = input("Вторая строка: ")

result = kmp_search(A, B)
print(f"Результат: {result}")
```